

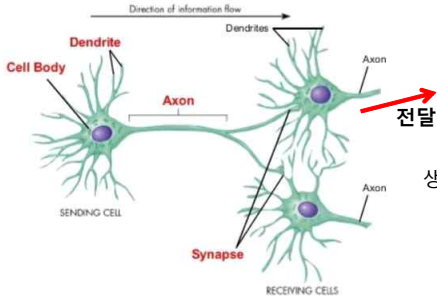
Chapter 10 .딥러닝 개념 및 심층신경망

- 딥러닝 개념
- 심층 신경망
- 활성화 함수

* **인공 신경망** (Artificial Neural Network, Neural Network)

- 딥러닝은 인간 신경망을 모방한 인공신경망의 한 종류로서 인공신경망 구조와 유사
- 수상돌기(Dendrite)를 통해 다른 뉴런에서 입력 신호를 받아 축색돌기(Axon)를 통해 다른 뉴런으로 신호를 전달
출력 신호는 입력된 신호가 모여서 일정한 용량을 넘어설 때 일어남

Neuron (Nerve cell) Anatomy

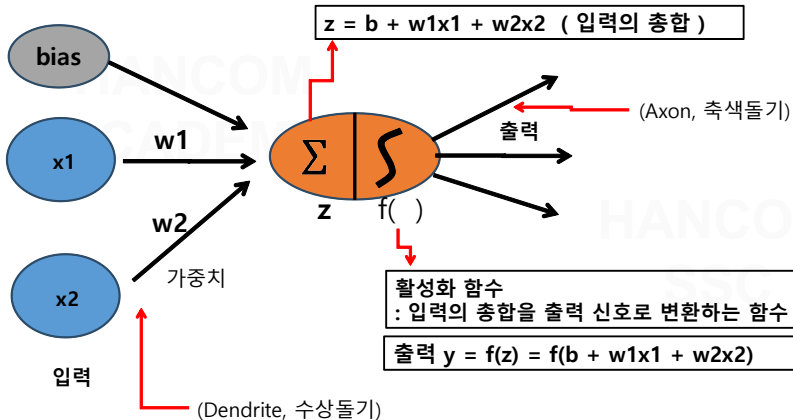


생물학적 신경세포
(뉴런)

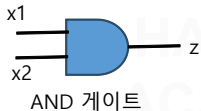
- 인공신경망 뉴런 모델은 생물학적 뉴런을 수학적으로 모델링
- 다른 여러개의 뉴런으로부터 입력값을 받아 세포체(Cell Body) 저장하다가 자신의 용량을 넘어서면 외부로 출력값을 보내는 것과 같이 인공신경망 뉴런은 여러 입력값을 받아서 일정 수준이 넘어서면 활성화되어 출력값을 내보냄

- 신경망 뉴런 모델링

: 생물학적 뉴런 모델을 수학적 모델링 (퍼셉트론)



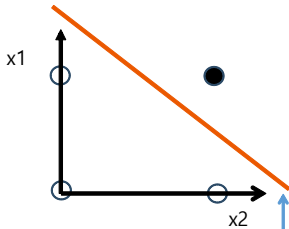
* 단일 퍼셉트론(AND)



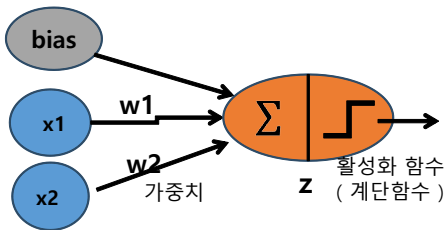
AND 진리표

x1	x2	z
0	0	0
0	1	0
1	0	0
1	1	1

진리표 좌표 평면 표현

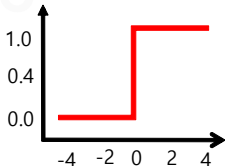


Z 결과값 1과 0을 구별할 수 있는 구분선(직선) 가능



$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(z) = \begin{cases} 0 & (z \leq 0) \\ 1 & (z > 0) \end{cases}$$



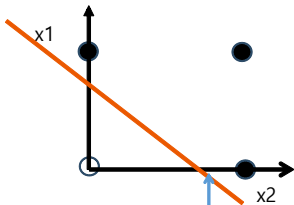
* 단일 퍼셉트론(OR)



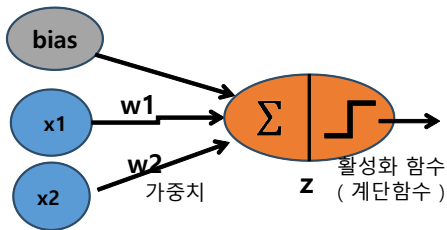
AND 진리표

x1	x2	z
0	0	0
0	1	1
1	0	1
1	1	1

진리표 좌표 평면 표현

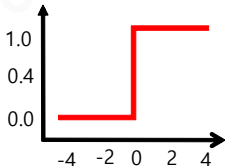


Z 결과값 1과 0을 구별할 수 있는 구분선(직선) 가능



$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(z) = \begin{cases} 0 & (z \leq 0) \\ 1 & (z > 0) \end{cases}$$



* 단일 퍼셉트론(AND, OR 코드 구현)

```
import numpy as np
```

```
def AND_Percept(x):
```

```
    w = np.array([1, 1])
```

```
    b = -1
```

```
    y = np.sum(w * x) + b
```

```
    if y <= 0:
```

```
        return 0
```

```
    else:
```

```
        return 1
```

AND

적용 가중치와 편향

$$y = w_1x_1 + w_2x_2 + b$$

```
def OR_Percept(x):
```

```
    w = np.array([0.5, 0.5])
```

```
    b = -0.2
```

```
    y = np.sum(w * x) + b
```

```
    if y <= 0:
```

```
        return 0
```

```
    else:
```

```
        return 1
```

OR

적용 가중치와 편향

$$y = w_1x_1 + w_2x_2 + b$$

* 단일 퍼셉트론(AND, OR 코드 구현)

```
if __name__ == '__main__':  
    print('AND Percept')  
    for x in [(0,0),(0,1),(1,0),(1,1)]:  
        y = AND_Percept(x)  
        print('입력 값 : ' + str(x) + ' ==> 출력 값 : ' + str(y))  
  
    print('OR Percept')  
    for x in [(0, 0), (0, 1), (1, 0), (1, 1)]:  
        y = OR_Percept(x)  
        print('입력 값 : ' + str(x) + ' ==> 출력 값 : ' + str(y))
```



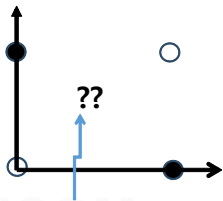
AND Percept	
입력 값 : (0, 0)	==> 출력 값 :0
입력 값 : (0, 1)	==> 출력 값 :0
입력 값 : (1, 0)	==> 출력 값 :0
입력 값 : (1, 1)	==> 출력 값 :1
OR Percept	
입력 값 : (0, 0)	==> 출력 값 :0
입력 값 : (0, 1)	==> 출력 값 :1
입력 값 : (1, 0)	==> 출력 값 :1
입력 값 : (1, 1)	==> 출력 값 :1

* 단일 퍼셉트론의 한계 (XOR 문제)



XOR 진리표

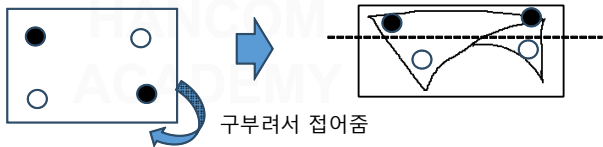
x1	x2	z
0	0	0
0	1	1
1	0	1
1	1	0



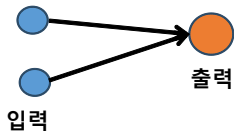
결과값 1과 0을 구별할 수 있는
구분선(직선)을 그을 수 없음!!

* 단일 퍼셉트론의 한계 (XOR 문제)

평면(공간)을 왜곡시켜 문제 해결 접근

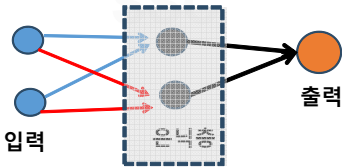


단일 퍼셉트론 구조

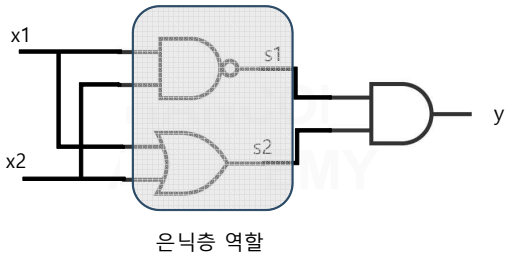


은닉층을 두어
공간 왜곡

다층 퍼셉트론으로 발전

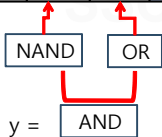


* NAND, OR , AND게이트 조합으로 구현한 XOR 게이트



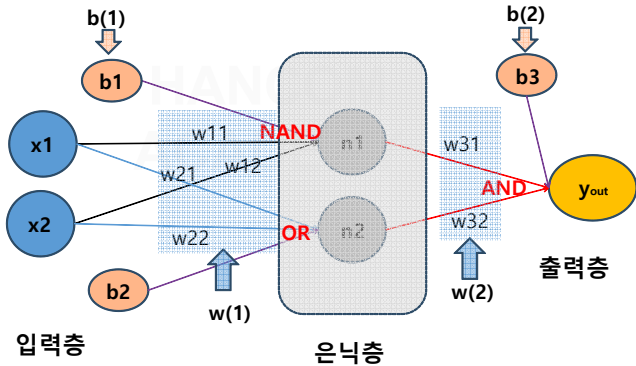
XOR 진리표

x1	x2	s1	s2	y
0	0	1	0	0
0	1	1	1	1
1	0	1	1	1
1	1	0	1	0



목표 값

* XOR 게이트의 다층 퍼셉트론 설계



XOR 문제 **정답을 위한**
임의의 가중치 지정

$w(1)$ 가중치 배열

$$= \begin{bmatrix} w11 & w21 \\ w12 & w22 \end{bmatrix} = \begin{bmatrix} -2 & 2 \\ -2 & 2 \end{bmatrix}$$

$w(2)$ 가중치 배열

$$= \begin{bmatrix} w31 \\ w32 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$b(1)$ 편향 배열

$$= \begin{bmatrix} b1 \\ b2 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

$b(2)$ 편향 배열

$$= b3 = \begin{bmatrix} -1 \end{bmatrix}$$

* 다층 퍼셉트론(XOR 코드 구현)

```
import numpy as np
```

```
w11_12 = np.array([-2, -2])
```

```
w21_22 = np.array([2, 2])
```

```
w3 = np.array([1, 1])
```

```
b1 = 3
```

```
b2 = -1
```

```
b3 = -1
```

```
def Percept(x, w, b):
```

```
    y = np.sum(w * x) + b
```

```
    if y <= 0:
```

```
        return 0
```

```
    else:
```

```
        return 1
```

XOR 문제 **정답을 위한**
임의의 가중치 직접 지정


차후 경사하강 + 오차역전파
로 가중치 자동 업데이트

가중치곱합 + 편향 결과의
따른 출력값

* 다층 퍼셉트론(XOR 코드 구현)

```
def NAND(x1, x2):  
    return Percept(np.array([x1, x2]),w11_12,b1)  
  
def OR(x1, x2):  
    return Percept(np.array([x1, x2]),w21_22,b2)  
  
def AND(x1, x2):  
    return Percept(np.array([x1, x2]),w3,b3)  
  
def XOR(x1, x2):  
    return AND(NAND(x1, x2),OR(x1,x2))
```

* 다층 퍼셉트론(XOR 코드 구현)

```
if __name__ == '__main__':  
    print('NAND Percept')  
    for x in [(0,0),(0,1),(1,0),(1,1)]:  
        y = NAND(x[0],x[1])  
        print('입력 값 : ' + str(x) + ' ==> 출력 값 : ' + str(y))  
    print('OR Percept')  
    for x in [(0, 0), (0, 1), (1, 0), (1, 1)]:  
        y = OR(x[0], x[1])  
        print('입력 값 : ' + str(x) + ' ==> 출력 값 : ' + str(y))  
    print('XOR Percept')  
    for x in [(0, 0), (0, 1), (1, 0), (1, 1)]:  
        y = XOR(x[0], x[1])  
        print('입력 값 : ' + str(x) + ' ==> 출력 값 : ' + str(y))
```

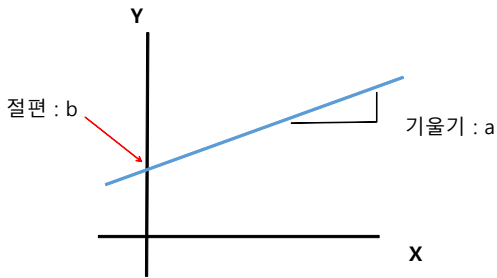
XOR Percept

입력 값 : (0, 0) ==> 출력 값 :0
입력 값 : (0, 1) ==> 출력 값 :1
입력 값 : (1, 0) ==> 출력 값 :1
입력 값 : (1, 1) ==> 출력 값 :0

* 선형회귀로 알아 보는 경사하강법

선형회귀

- 데이터를 선형 상관 관계로 모델링해 우리가 알고자 하는 값을 예측하는 방식
- 예) 공부량과 성적, 몸무게 와 키 상관 관계 예측
- 관찰된 데이터를 기반으로 가장 거리가 짧은 데이터(손실이 가장 적은)를 대표할 수 있는 선을 긋는 것이며, 그 선을 회귀식이라고 함
- 회귀식 가설함수 $H(x) = Wx + b$ (w : 가중치, b : 편향)
 - : 기울기 W , 절편 b 에 따라 선의 모양이 정해지기 때문에 **관찰된 데이터와 오차가 가장 적은 W, b 를 구함**으로써 새로운 데이터에 대한 예측을 수행할 수 있음



$$Y = a * X + b$$

(1차 방정식)



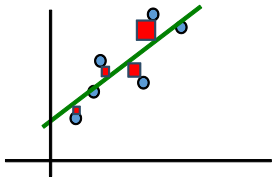
기계학습

$$Y(\text{예측}) = Wx + b$$

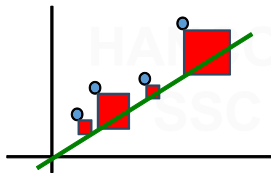
훈련과정은 모델 파라미터인
최적의 기울기(가중치)와 절편
을 찾는 과정

* 오차함수와 비용함수

- 입력 데이터에 대해 예측값이 타겟에 최대한 근접하도록 가중치(기울기) 벡터를 최적화하는 과정이 선형회귀 학습의 목표
- 예측값과 타겟(라벨)의 오차를 근거로 가중치를 조절하기 때문에 오차를 관리하고 변화를 추적



Vs



가중치(기울기)에 따른 오차 크기 비교

* 오차 함수

- 타겟(라벨)값 과 예측값의 차이에 대한 제곱을 사용

$$\text{Error Function} = (\hat{y}^{(i)} - y^{(i)})^2$$

$\hat{y}^{(i)}$: i 번째 데이터의 **예측 값**

$y^{(i)}$: i 번째 데이터의 **라벨(타겟)** $\hat{y}^{(i)} = Wx_i + b$

* 선형회귀에서 비용 함수(Error Function, Loss Function)

: 예측값과 실제 값의 오차가 가장 작아지는 해(w,b)를 찾는 목적으로 **평균제곱오차(MSE)** 함수 사용

$$\text{Mean Square Error(평균제곱오차)} = \frac{1}{n} \sum_{i=0}^n ((Wx_i + b) - y_i)^2$$

회귀함수($Wx_i + b$) 와 실제 데이터 사이의 거리(오차)를 제곱한 값을 평균낸 값



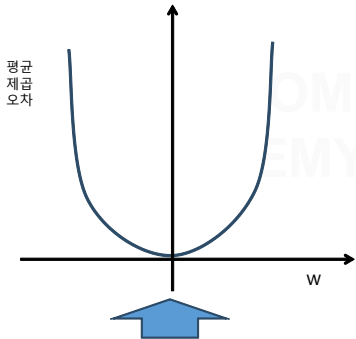
선형 회귀의 목적 :

평균제곱오차를 최소화하는

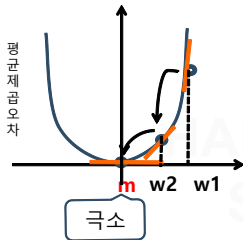
w, b를 찾아가는 작업이라 정의

* MSE : 여러 손실(비용)함수 중의 하나

* 오차 수정을 위한 경사 하강법 (가중치 갱신)



기울기(가중치) w 와 오차와의 관계를 표현한 그래프
 : 가설함수의 기울기가 양의 값으로 너무 커져도 실제 데이터의 분포와 떨어져 손실(오차)는 무한대로 커지고
 반대로 가설함수의 기울기가 음의 값으로 너무 커져도 실제 데이터의 분포와 떨어져 손실(오차)는 무한대로 커짐
 따라서, 실제 데이터 분포와 오차가 가장 작은 지점의 가설함수의 기울기(w)를 찾아야 함

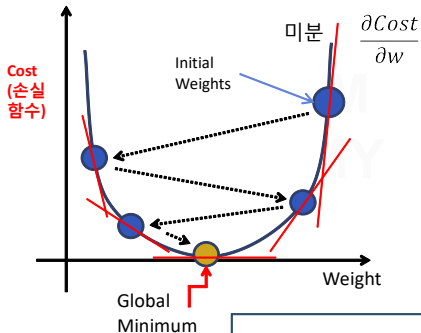


- 가중치를 $w_1 \rightarrow w_2 \rightarrow m$ (극소)로 이동시켜 오차 최소점을 찾아야함.
- 순간기울기(미분)를 이용한 경사 하강법 활용
- 오차 최소지점(m)의 순간기울기(미분)은 0
- 즉, 미분 값이 0인 지점을 찾는 것

MSE(평균제곱오차) 비용함수의 극소(최저점)를 갖는 기울기(w)의 가설함수를 구하는것이 목표

* 오차 수정을 위한 경사 하강법 (가중치 갱신)

w에 대한 비용함수
Cost(W) 그래프에서
w 위치의 순간 기울기(미분)
계산



- Initial Weights(가중치)에 대한 미분 계수 구함
- 기울기(미분 계수)가 양의 값(+) 이면 음의 방향으로 Weight(가중치) 감소
- 기울기(미분 계수)가 음의 값(-) 이면 양의 방향으로 Weight(가중치) 증가
- 이동 시킨 Weight(가중치)로 다시 미분 계수를 구하고 미분 값이 0이 아니면 위 과정을 반복
- 결국, 기울기가 0인 한 점으로 수렴(Global Minimum)

경사하강법 : 위 처럼 반복적으로 기울기(가중치)를 변화 시켜서 손실(오차) 최소 지점을 찾는 방법을 말함.

수식화)

$$W^+ = W - \alpha \frac{\partial MSE(w,b)}{\partial w}$$

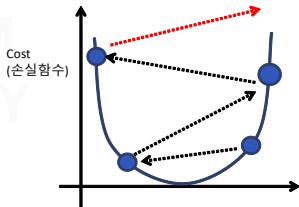
$$b^+ = b - \alpha \frac{\partial MSE(w,b)}{\partial b}$$

- W : 가중치 벡터
- α : 학습률(learning rate)
: 가중치 갱신 비율을 조정
- b : 편향

* 학습률 (Learning rate)

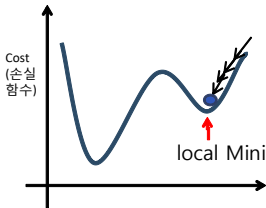
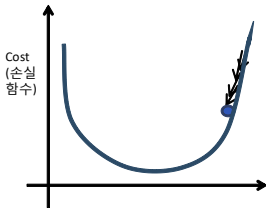
- 한 번의 학습으로 **가중치 값을 얼마나 갱신하지를 정하는 하이퍼파라미터** (가중치 이동 크기)
- 초기 0.01 전후로 지정하여 학습 시도를 권함

Learning
Rate가 너무
클 경우



- 최적화된 W 값을 지나쳐 학습이 제대로 이루어지지 않고 오히려 손실함수 값이 더 커지는 오버슈팅(overshooting) 현상 발생

Learning
Rate가 너무
작을 경우



- 학습의 성능이 떨어지거나,
- 다른 비용(손실)함수 적용 시 국소 최적치(Local Minimum)에 빠져 최적화된 W 값을 찾기도 전에 학습이 종료될 수 있음

* MSE 비용함수 미분을 위한 기초 미분 공식

미분(순간기울기) : x 값 변화량에 따른 y 값 변화량

1. $f(x) = a$ 에서 a 가 상수 일 때 미분 값은 0 , $\frac{d}{dx}a = 0$

2. $f(x) = x$ 일때 미분 값은 1 , $\frac{d}{dx}f(x) = 1$

3. $f(x) = ax$ 일때 a 가 상수이면 미분 값은 a

4. $f(x) = x^a$ 에서 a 가 자연수이면 미분 값은 ax^{a-1}

5. $\frac{d}{dx}(f(x) + g(x)) = \frac{d}{dx}f(x) + \frac{d}{dx}g(x)$

6. $\frac{d}{dx}f(g(x)) = f'(g(x))g'(x)$, 체인 룰

* 편 미분

$$f(x, y) = x^2 + yx + a \quad (a \text{는 상수})$$

$\frac{df}{dx}$ ← 함수 f 를 x 에 대해 편 미분

$$\begin{aligned} f(x, y) &= x^2 + yx + a \\ \frac{df}{dx} &= \overset{\textcircled{4}}{2x} + \overset{\textcircled{3}}{y} + \overset{\textcircled{1}}{0} \\ &= 2x + y \end{aligned}$$

* MSE 비용함수 편미분

▪ w(가중치) 편미분

$$\begin{aligned}\frac{\partial MSE(w,b)}{\partial w} &= \frac{1}{n} \sum [(wx^{(i)} + b - y^{(i)})^2]' \\ &= \frac{2}{n} \sum (wx^{(i)} + b - y^{(i)})[(wx^{(i)} + b - y^{(i)})]' \\ &= \frac{2}{n} \sum (wx^{(i)} + b - y^{(i)}) x^{(i)}\end{aligned}$$

← 체인 룰 적용

▪ b(편향) 편미분

$$\begin{aligned}\frac{\partial MSE(w,b)}{\partial b} &= \frac{1}{n} \sum [(wx^{(i)} + b - y^{(i)})^2]' \\ &= \frac{2}{n} \sum (wx^{(i)} + b - y^{(i)})[(wx^{(i)} + b - y^{(i)})]' \\ &= \frac{2}{n} \sum (wx^{(i)} + b - y^{(i)}) * 1\end{aligned}$$

← 체인 룰 적용

* MSE 비용함수 편미분 - 파이썬 코드 설계

▪ w(가중치), b(편향) 편미분 코드

```
y_pred = w * x_data + b # y = wx + b 예측 가설 함수 활용한 예측치  
error = y_data - y_pred # 실제값 - 예측치, 즉 오차 값
```

평균 제곱 오차를 w로 편미분한 결과

```
w_diff = -(2 / len(x_data)) * sum( (error) * x_data )
```


$$\sum (y^{(i)} - wx^{(i)} + b) x^{(i)}$$

평균 제곱 오차를 b로 편미분한 결과

```
b_diff = -(2 / len(x_data)) * sum(error)
```

▪ 가중치, 편향 업데이트(갱신) 코드

```
w = w - lr * w_diff # 미분 결과에 학습률을 곱한 후 기존 w 값 갱신  
b = b - lr * b_diff # 미분 결과에 학습률을 곱한 후 기존 b 값 갱신
```


* 경사하강법 활용 - 선형회귀 전체코드

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 몸무게 x 와 키 y 데이터 셋 준비

x_data = np.array([0.65, 0.70 , 0.75, 0.80]) # 연산을 위해 몸무게 스케일 조정
y_data = np.array([160, 175, 170, 180])

# 기울기(가중치) w 와 절편(편향) 값 초기화
w = 0
b = 0
# 학습률 지정
lr = 0.03
```

반복 횟수 지정

epochs = 110001 # 가중치와 편향 변동 없는 지점 체크해 에폭 설정함

경사하강법 시작

for i in range(epochs):

y_pred = w * x_data + b # $y = wx + b$ 예측 가설 함수 활용한 예측치

error = y_data - y_pred # 실제값 - 예측치, 즉 오차 값

평균 제곱 오차를 w로 편미분한 결과

w_diff = -(2 / len(x_data)) * sum((error) * x_data)

평균 제곱 오차를 b로 편미분한 결과

b_diff = -(2 / len(x_data)) * sum(error)

w = w - lr * w_diff # 미분 결과에 학습률을 곱한 후 기존 w 값 갱신

b = b - lr * b_diff # 미분 결과에 학습률을 곱한 후 기존 b 값 갱신

if i % 100 == 0: # 100회 마다 갱신된 가중치, 편향 출력

print('epoch = %d , 가중치 = %.4f , 편향 = %.4f' % (i, w, b))

epoch = 0, 가중치 = 7.4700, 편향 = 10.2750

epoch = 100, 가중치 = 81.8563, 편향 = 111.9355

epoch = 200, 가중치 = 82.2048, 편향 = 111.6929

epoch = 300, 가중치 = 82.5441, 편향 = 111.4464

epoch = 400, 가중치 = 82.8792, 편향 = 111.2029

.....

.....

epoch = 109800, 가중치 = 110.0000, 편향 = 91.5000

epoch = 109900, 가중치 = 110.0000, 편향 = 91.5000

epoch = 110000, 가중치 = 110.0000, 편향 = 91.5000

for 반복문 수행 이후

$y_{pred} = w * x_data + b$ # 위에서 구한 최적 w, b 로 데이터 예측

print("최적해 방정식 적용 예측 : ", y_{pred})

plt.scatter($x_data * 100, y_data, c='red'$) # 실 데이터

plt.plot($x_data * 100, y_{pred}$) # 예측 회귀 직선 그래프

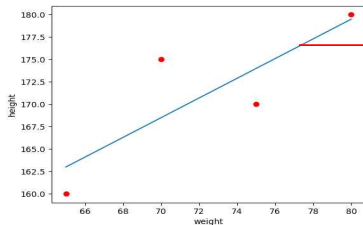
plt.xlabel('weight')

plt.ylabel('height')

plt.show()

최적해 방정식 적용 예측 : [163.00000296 168.50000102 173.99999909 179.49999716]

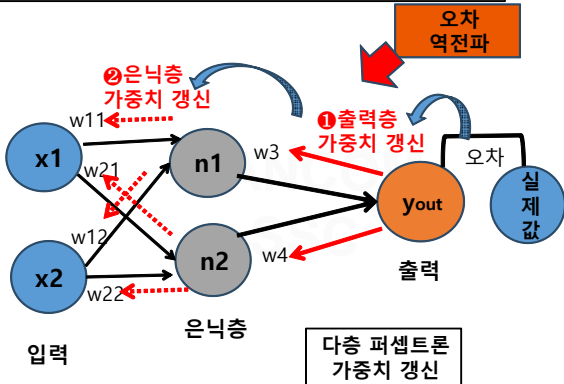
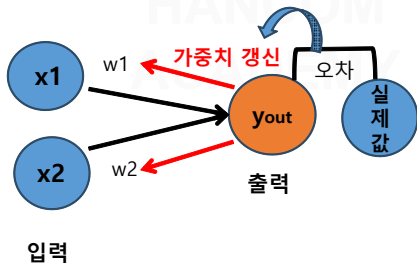
실제값 : [160, 175, 170, 180]



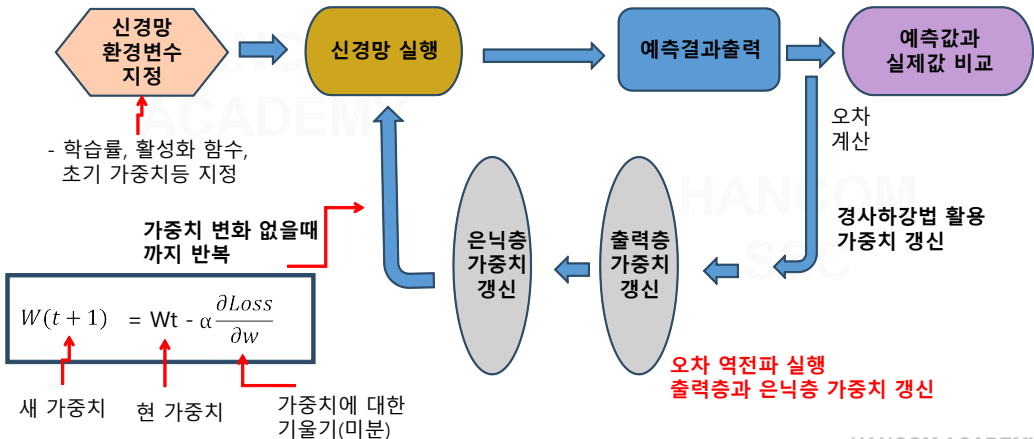
예측 회귀선

* 은닉층 가중치 갱신을 위한 오차 역전파

- 다층 퍼셉트론에서 입력층 < 은닉층 < 출력층 처럼 역순으로 거슬러 올라가면서 오차를 전파해 가중치를 차례로 갱신(조정)해 나가는 최적화 과정



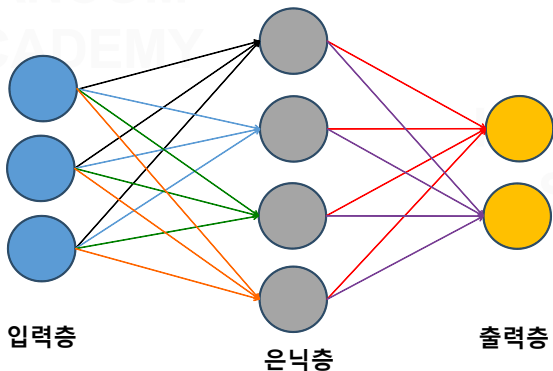
* 오차 역전파 신경망 구조



오차 역전파 원리 및 계산 별첨 자료 참조

- 인공 신경망 (뉴럴 네트워크)

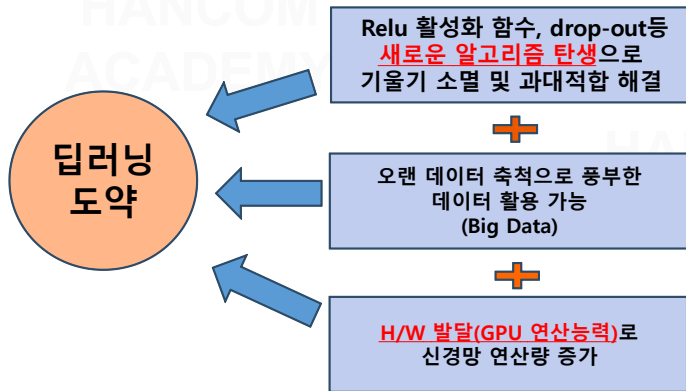
: 수학적 모델링된 뉴런을 여러개 쌓아 설계한 다층 퍼셉트론(Multi-Layer Perceptron, MLP) 구조로서 여러 퍼셉트론이 서로 연결되어 있는 구조의 네트워크



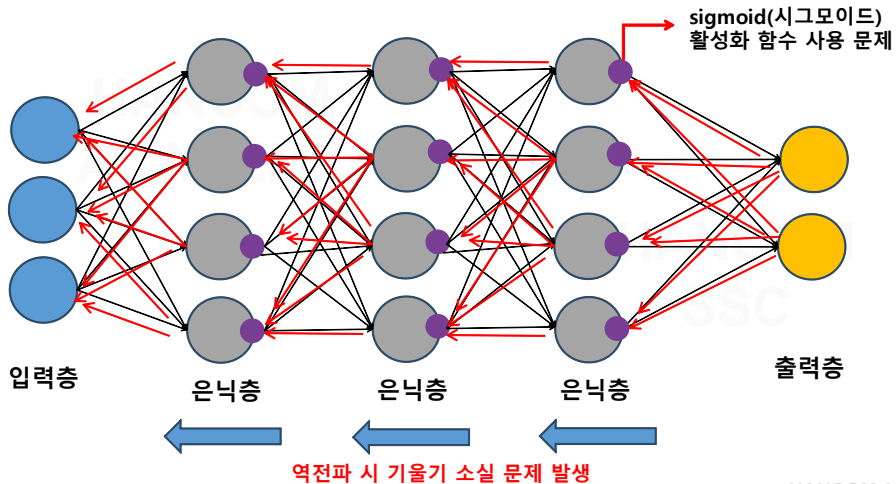
* 다층 퍼셉트론(Multi-Layer Perceptron, MLP) 구조 **한계**

- 문제 해결을 위해 여러 은닉층을 쌓을 경우 오차역전파 학습 과정에서 기울기가 사라져 학습이 잘 되지 않은 현상, 경사도 소멸(Vanishing Gradient) 문제 발생
(은닉층 활성화 함수로 sigmoid 활성화 함수 사용에 대한 문제)
- 학습한 내용에 대해서는 성능이 뛰어나나 새로운 데이터 예측에 대해서는 성능이 현저히 떨어지는 현상 (과대적합 문제)
- 은닉층이 증가할 수록 연산량이 기하급수적으로 급증하는 문제

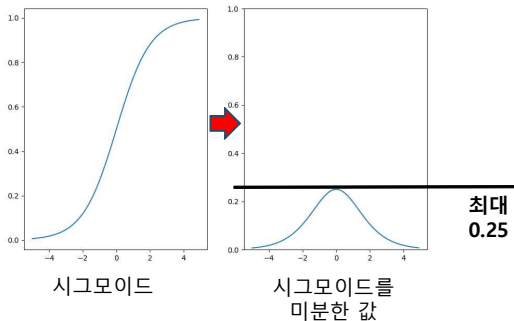
- * 딥러닝 - 인공신경망(다층퍼셉트론)의 여러 한계를 극복하여 문제점을 해결한 알고리즘으로 인공신경망의 새이름!
- 인공심층신경망(Artificial Deep Neural Network)



* 다층 퍼셉트론(신경망) 기울기 소멸과 활성화 함수



* Sigmoid(시그모이드) 함수 미분



* 역전파 시 시그모이드 활성화 함수를 사용한 여러 은닉층을 통과할때 기울기(미분)값이 1 보다 작은 최대 0.25 값을 갖음으로 계속 곱하다 보면 결국 0에 가까워져 가중치를 수정하기 어려워지게되는 문제 발생!!

$$\text{시그모이드 함수 : } y = \frac{1}{1 + \exp(-x)}$$

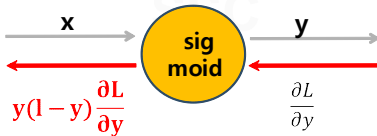


미분

시그모이드 노드 로컬 그래디언트 :

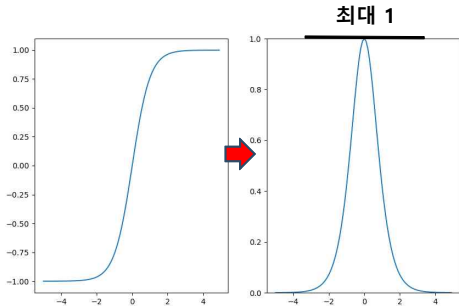
$$\frac{\partial y}{\partial x} = y(1 - y)$$

최대 0.25 값을 갖음



시그모이드 활성화함수 역전파

* 활성화 함수 발전



하이퍼블릭
탄젠트 함수

하이퍼블릭
탄젠트 미분값

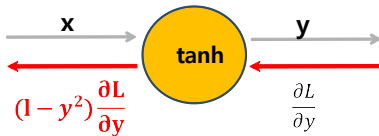
* 하이퍼블릭 탄젠트 함수

$$f(x) = \tanh(x)$$

시그모이드 함수의 범위를 -1 ~ 1로 확장한 개념

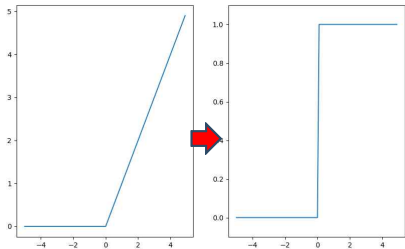
미분값의 범위도 함께 최대 1로 확장되는 효과

하지만, 여전히 1보다 작은 값이 존재하므로
기울기 소멸 문제는 계속 존재!!



* Relu 활성화 함수 (기울기 소멸 해결)

은닉층 활성화함수로 대부분 사용



Relu 함수

Relu 미분값

* Relu(렐루) 함수

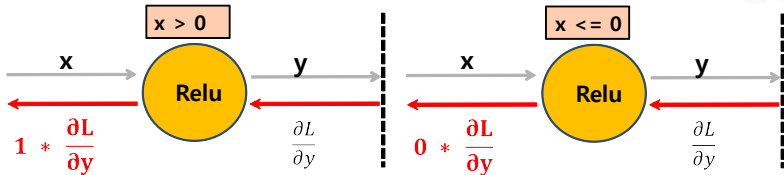
: x가 0보다 작을때 0으로 처리, 0보다 큰 값은 x를 그대로 사용

$$y = x \ (x > 0)$$

$$y = 0 \ (x \leq 0)$$

Relu 노드의 로컬 그래디언트 :

$$\frac{\partial y}{\partial x} = 1 \ (x > 0) \quad , \quad \frac{\partial y}{\partial x} = 0 \ (x \leq 0)$$



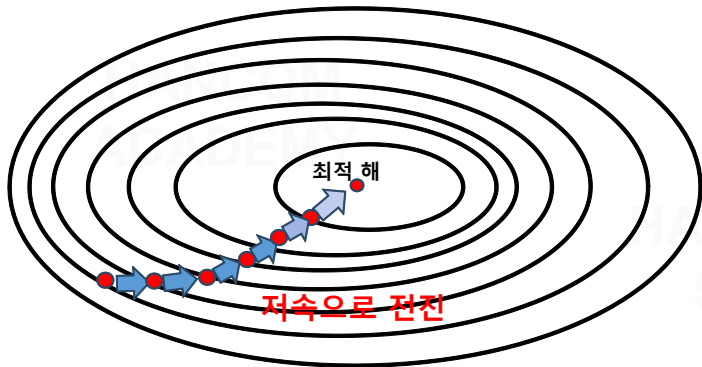
➤ x가 0보다 크기만 하면 미분값이 1이 되어 여러 은닉층을 통과해 곱해져 더라도 기울기는 소멸되지 않음!!

■ 최적해 찾는 속도와 정확도 해결을 위한 고급 경사하강법

1. 배치 경사하강법 (BGD : Batch Gradient Descent)

- 전체 학습 데이터를 하나로 배치로 묶어 학습시키는 경사하강법
- 전체 데이터에 대한 오차의 평균 계산, 미분을 통해 기울기 산출 최적화 진행
- 전체 데이터를 통한 학습으로 가장 가중치 갱신 회수가 적음
(1 Epoch 당 1회 갱신)
- 전체 데이터를 모두 한 번에 처리하기 때문에 메모리 사용이 많음
- 전체 데이터에 대한 기울기를 구하기 때문에, 수렴에 안정적 이나 학습 속도는 가장 느림

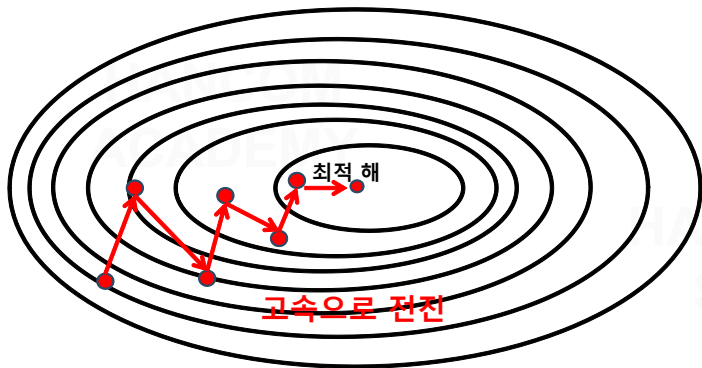
배치 경사하강법 시각화



2. 확률적 경사하강법 (SGD : Stochastic Gradient Descent)

- 단 하나의 데이터를 랜덤 추출 하여 경사 하강법을 1회 진행하는 방법 (배치 size : 1)
- 배치 경사 하강법에 비해 적은 데이터로 학습할 수 있고, 속도가 빠른 장점
- 지역 최저점에 빠질 확률을 줄여 주지만 진폭이 크고 불안정하여 Global 최저점에 수렴하기는 어려움
- 각 데이터에 대한 손실값의 기울기는 약간씩 다르기 때문에 손실의 평균이 아닌 개별 데이터에 대한 미분을 수행하면 기울기의 방향이 매번 크게 바뀜

확률적 경사하강법 시각화



3. 미니배치 확률적 경사하강법

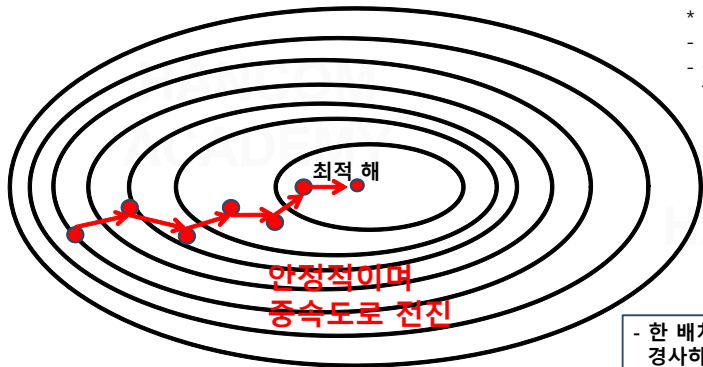
(MSGD : Mini-Batch , Stochastic Gradient Descent)

전체 데이터셋을 모델에 통과시켜 훈련시키는 방법이 아닌,
랜덤샘플링한 k개의 샘플을 모델에 통과시켜 손실함수로
손실을 구하고, 미분(기울기) 활용한 손실 최점을 향하도록 가
중치를 업데이트하는 방법

샘플링과정은 **비복원 샘플링(추출)**로서 한번 학습에 사용된 데
이터는 모든 데이터가 학습에 사용될때 까지 사용되지 않음.

SGD와 BGD의 절충안으로, 전체 데이터를 batch_size 개씩
나눠 학습(배치 크기 사용자 지정)시키는 방법

미니배치 확률적 경사하강법 시각화

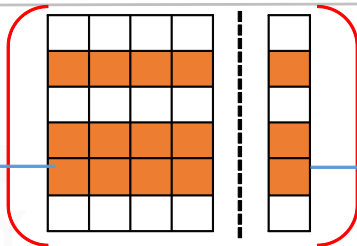


- * 전체 데이터 1000개 학습 시킬 경우
- batch_size : 100
- 전체를 100개씩 총 10묶음으로 배치
1 Epoch 당 10번 경사하강 진행

- 한 배치에 대한 손실값의 평균으로 경사하강 진행하기 때문에 Shooting이 적고, Local Minimum에 빠질 확률을 어느정도 줄일 수 있음
- BGD보다 계산량이 적음

▶ 1회 가중치 갱신 도식화

Data Set =



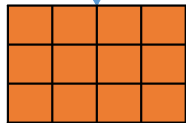
훈련데이터
특성

타깃데이터

랜덤 샘플링
(확률적 의미)

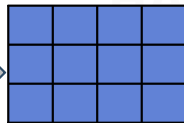
랜덤 샘플링

k개 샘플



가중치

모델



예측 출력값

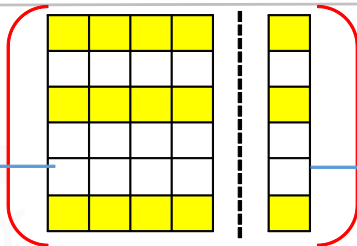


타깃 실제값

Loss 함수
손실 평균값에 대한 경사하강 가중치 갱신

▶ 2회 가중치 갱신 도식화

Data Set =



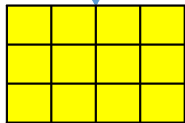
훈련데이터
특성

타깃데이터

랜덤 샘플링
(확률적 의미)

랜덤 샘플링

k개 샘플



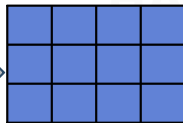
(미니배치 의미)



가중치



모델



예측 출력값



타깃 실제값

Loss 함수
손실 평균값에 대한 경사하강 가중치 갱신

- 샘플링 과정이 비복원 추출로서 더 이상 모델을 통과하지 않은 샘플들이 없을 때까지 기존에 모델을 통과한 샘플들은 사용되지 않음
- 샘플들이 전부 소진되면 다시 전체 샘플들에 대해서 비복원 추출이 진행
- **1 Epoch(에포크)** : 전체 데이터셋의 샘플들이 모두 모델을 통과하여 **전체 데이터에 대한 한번의 훈련을 마친 상태**
- **1 iteration(이터레이션)** : 한 개의 미니배치를 모델에 통과시켜 모델 **가중치 업데이트를 마친 상태**

경사하강법 정리

- BGD(배치 경사하강)

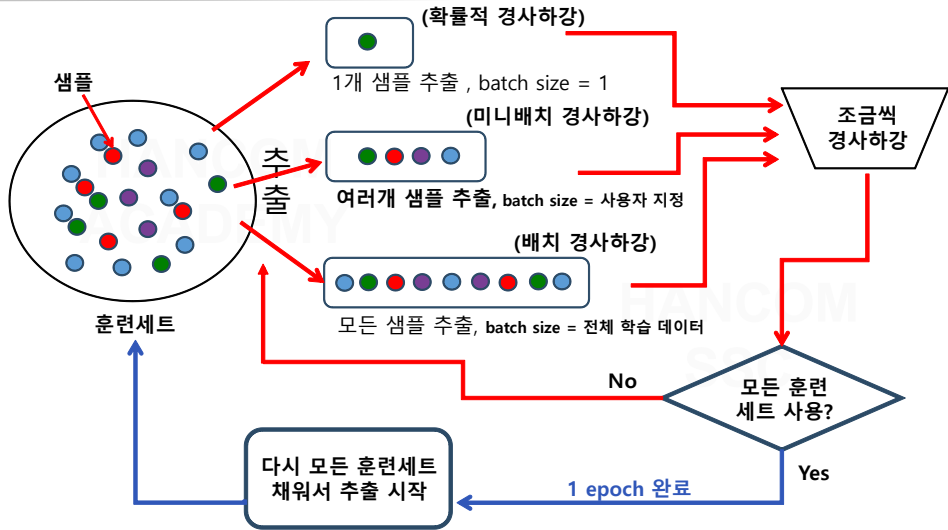
: 전체 데이터를 모두 사용해 기울기 계산, 가중치 갱신
안정적인 방법이지만 하나 전체 데이터 사용으로 계산량이
많고 학습이 오래 걸림

- SGD(확률적 경사하강)

: 한개의 샘플을 무작위로 선택, 하나의 샘플에 기울기 계산 후
가중치 갱신, 배치 크기가 1인 경사하강

- MSGD(Mini-Batch SGD, 미니배치 경사하강)

: 랜덤 샘플링한 K 샘플을 묶어 기울기 계산 후 가중치 갱신,
보통 256, 512 배치 사이즈 설정

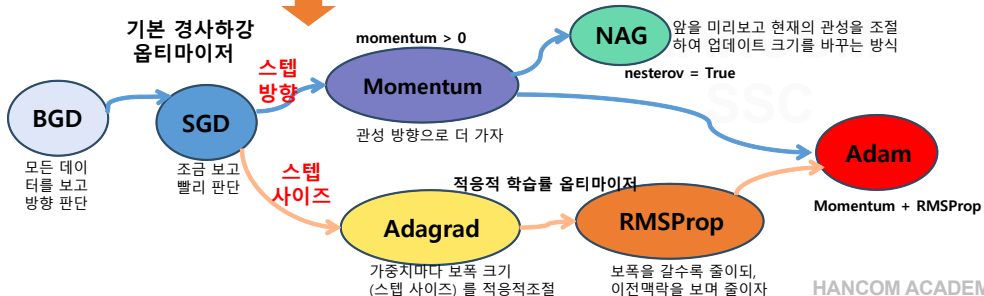


■ 학습 최적화를 위한 옵티마이저(Optimizer)

- 학습 최적화 : 손실 함수(loss function)의 최솟값을 찾아가는 일련의 과정
 - : 역전파 과정 중에 가중치를 업데이트(경사하강)하면서 진행되며 이때 한 스텝마다 이동하는 크기, 즉 보폭은 학습률(learning rate)로 정의되고 앞으로 이동할 방향은 현 지점의 기울기(gradient) 통해 정의

기본 경사하강 옵티마이저 (SGD) 경우

: 학습이 빠르게 이뤄진다는 장점이 있지만, 지그재그로 핑퐁하면서 불안정한 학습 과정을 보임
이를 개선한 다양한 옵티마이저 등장



* 모멘텀 (Momentum , 가속도)

- 학습 방향을 유지하려는 성질
- 모멘텀을 사용하면 학습 방향이 바로 바뀌지 않고, 일정한 방향을 유지하여 움직임
- 매 스텝마다 gradient(기울기) 가 동일한 부호(같은 방향)을 가지면 관성변수를 계속 증가시켜 즉, 같은 방향으로 이동할 수록 더 많이 이동하게 하여 학습의 가속화 기대
- 가중치가 local minimum에 빠지지 않고 학습하는 방향으로 더 가도록 하는 방법
- SGD 클래스에서 momentum = 0.0 (default), $0 < \text{momentum} < 1$ 범위로 설정 가능
보통 0.5 ~ 0.9 사이값으로 설정

* 적응적 학습률 (Adagrad, RMSProp)

: 가중치 매개변수 원소마다 **적응적으로 학습률 갱신 정도를 조정**, 학습률 보폭 크기 개선 알고리즘

Adagrad

- 학습을 통해 **크게 변동이 있었던 가중치에 대해서는 학습률 감소**
변화가 많았던 변수들은 최적 지점 근처에 있을 확률이 높기 때문에 learning rate을 작게 함으로써 더욱 세밀하게 Update
- 학습을 통해 아직 **가중치의 변동이 별로 없었던 가중치는 학습률을 증가** 시킴
변화가 적었던 변수들은 최적 지점에 멀리 벗어나 있을 확률이 높기 때문에 learning rate를 크게 함으로써 최적 지점에 수렴하도록 만듦

RMSProp

- AdaGrad 단점을 보완한 기법, AdaGrad는 무한히 학습한다면 갱신량이 0이 되어 전혀 갱신되지 않는 일이 발생, 이를 개선한 것이 RMSProp
과거의 기울기는 조금 반영하고 최신의 기울기를 많이 반영(이전 맥락 반영)
하여 학습률 크기 민감도 개선
keras compile()메서드 기본 Optimizer = RMSProp

* Adam 옵티마이저

- Momentum 과 AdaGrad 를 융합한 기법
- 학습률, 일차 모멘텀용 계수, 이차 모멘텀용 계수로 3개의 hyperparameter를 설정
- 일차 모멘텀용 계수 = 0.9, 이차 모멘텀용 계수=0.999를 기본 설정값으로 하면 보통 좋은 성능 발휘
- 현재 가장 많이 사용되는 옵티마이저

* 케라스(keras) 옵티마이저 활용법

옵티마이저	내용	효과	케라스 사용법
확률적경사하강 (SGD)	랜덤 추출한 하나의 데이터를 사용해 가중치 갱신, 더 빨리 자주 업데이트	속도 개선	keras.optimizers.SGD(lr = 0.1) default lr : 0.01
모멘텀 (Momentum)	관성의 방향을 고려해 방향을 유지, 가속	정확도 개선	keras.optimizers.SGD(lr = 0.1, momentum = 0.9) #모멘텀 계수 추가
네스테로프 모멘텀 (NAG)	모멘텀이 이동시킬 방향으로 미리 이동해서 그레이디언트 계산, 스텝 이동	정확도 개선	keras.optimizers.SGD(lr = 0.1, momentum = 0.9, nesterov = True) #네스테로프 옵션 추가
아다그라드 (Adagrad)	가중치 매개변수 별 변동성에 따라 학습률을 적응적으로 조절하는 방법	보폭 크기 개선	keras.optimizers.Adagrad(lr = 0.01)
알엠에스프롭 (RMSProp)	Adagrad 보폭 크기를 이전 맥락에 따라 조절, 보폭 민감도 보완	보폭 크기 개선	keras.optimizers.RMSprop(lr = 0.01)
아담 (Adam)	모멘텀과 알엠에스프롭 방법을 융합한 방법	정확도와 보폭 크기 개선	keras.optimizers.Adam(lr = 0.001, beta_1=0.9, beta_2=0.999)

Thank You